

Hypit 小白指南

给 AI Agent 用的「视频语言 + 系统」—— 复刻爆款视频，产出的是可复用的 workflow

开源 18.9k★

TypeScript • v0.2.17

Apache-2.0 带附加条件

github.com/hypit-ai/hypit

2026-10 实测学习

Q1 是什么 Q2 对比剪辑软件 Q3 SVML 源文件 Q4 词锚点机制 Q5 流水线架构 Q6 生成模型可选
Q7 变体对账 Q8 CLI 命令 Q9 组件复用 Q10 词级对齐 Q11 许可证坑 Q12 上手路径 能力清单
动手实验×3

源头：Rachel 的实测帖——把「小Lin说」的参考视频连同 Hypit 丢给 Codex，Agent 产出的不只是一条成片，而是一套可以继续修改和复用的视频 Workflow。本指南基于官方仓库 v0.2.17 源码与 examples 逐文件解剖 + 3 个零依赖实验真跑验证。

疑问解答（疑问即目录）

Q1 Hypit 到底是什么？不是 Agent、不是视频模型，那它是什么？

官方定义一句话：**Hypit 给 Claude Code、Codex 这类编程 Agent 提供「制作视频的语言和系统」**。你把一条参考视频交给 Agent，它把视频“复刻”成一份完整的、纯文本的工作流——画面、台词、字幕、B-roll、特效全部变成可编辑的源文件，而不是剪映里钉死的时间线。

不是 Agent

不是视频模型

不是托管剪辑器

⇒ 是

一套「视频源语言 SVML」+「编译运行系统」

类比：视频界的 HTML+浏览器，Agent 来当“前端工程师”

三个“不是”与一个“是”——它把自己定位成 Agent 的创作底座，而非又一个黑盒工具

类比：就像网页不是 Photoshop 画出来的、而是 HTML 写出来的一样，Hypit 想让视频从“剪辑出来”变成 **“写出来 + 编译出来”**。写源文件的是 AI Agent，人只提要求：“照这条视频的样子，把产品换成我的”。

Q2

和剪映 / Premiere 差在哪？"锚定词不锚定秒"是什么意思？

传统剪辑里，一切素材都 **钉死在时间线的某一秒**：字幕从 3.5s 出现、贴图 5.2s 飞入。可一旦换配音、换语速、换语言，全片逐帧重排——这就是"改一条视频比做一条还累"的原因。Hypit 的核心思想是：事件锚定在**词**上——"说到 hair gel 这个词时弹出发胶贴图"，词到哪，画面跟到哪。

	剪映 / PR	Hypit (SVML)
载体	二进制工程文件 + 鼠标拖时间线	纯文本 .svml 源文件 (git 可管、diff 可读)
事件定位	绝对秒数 (钉死)	词级语义锚点 (跟着配音走)
改主持人	几乎重做全片	只重新生成主持人相关镜头 (实验2实测相似度 79%)
谁来干活	人手剪	AI Agent 读参考视频 → 写出整份工作流
批量变体	逐条复制修改	一条命令批量产出 100 个变体 (官方口径)

Q3

SVML 长什么样？一条 20 秒足球 Tier List 的源文件里都有什么？

SVML 是一种 XML 方言（官方叫 markup）。实验1解剖了官方示例 [ranking-football/reference.svml](#)（329 行）：顶部 声明 + **21 个 @hypit 官方能力包导入** + **1 个本地组件包**，正文 160 个元素——24 个生成节点（gpt:Image 图、asset:Audio 音、render:Video 成片）和 14 段 text:Value 图像提示词。

```
<?svml using="@hypit/markup@1"?>
<svml>
  <import as="gpt"      from="@hypit/gpt-image@1"/>  ← 生图能力包
  <import as="whisperx" from="@hypit/whisperx@1"/>   ← 词级对齐包
  <import as="ranking"  from="@hypit/ranking@1"/>     ← 排行榜组件包
  <import as="render"   from="@hypit/render-hyperframes@1"/>
  <import as="recipes"  source="./recipes.svs"/>      ← 本地外观包(svs)

  <script id="story">                                ← 台词 + 语义锚点
    <ronaldo>Ronaldo is <D | Dee> tier. || Bro has
      @{{hair-gel}} more hair gel than @{{trophy}} trophies ...</ronaldo>
  </script>

  <gpt:Image id="presenter" prompt={presenter-prompt} aspect-ratio="9:16"/>
  <render:Video id="final" .../>                     ← 成片目标
</svml>
```

✅ **实验1实测 (exp1_svml_anatomy.py 真实输出)**：reference.svml 共 8 个词锚事件全部配对成功、22 个 || 停顿、2 处读音可选，**硬编码秒数属性 = 0 个** ——“锚词不锚秒”在源文件层面是铁证。

Q4 @{事件}词@{/事件} 内联锚点是怎么变成画面的？ || 和 <D|Dee> 又是干嘛的？

Script 台词是全片的“语义总开关”。三种内联记号：

记号	作用	例子（来自 reference.svml）
@{event}词@{/event}	词级事件锚点：B-roll/字幕/特效挂在这几个词上	@{hair-gel} more hair gel than @{trophy} trophies——说到“发胶”弹发胶罐图，说到“奖杯”弹奖杯图
	停顿/换气标记：控制喜剧节奏（reference 里 22 个）	"Bro has more hair gel than trophies"
<D Dee>	读音可选：给配音模型指定怎么念（2 处）	<D Dee> tier、<S Ess> tier——念字母名不念单词

Bro has @{hair-gel} more hair gel than @{trophy} trophies || at this point.

hair-gel 事件 → 弹发胶罐 B-roll

trophy 事件 → 弹奖杯 B-roll

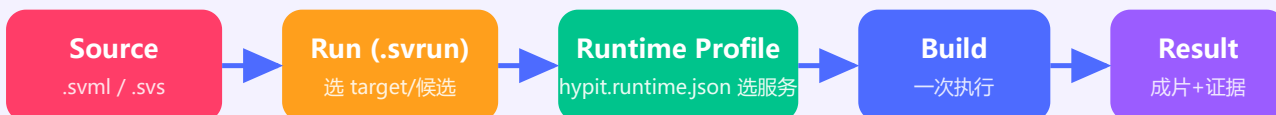
|| → 停顿 0.3~0.5s

配音念到哪个词，事件就在那个词的时刻触发 —— 词走画面走，永不脱节

台词里的两个锚点如何各挂一个 B-roll 事件

Q5 整个系统怎么跑起来？Source → Run → Runtime → Build → Result 是什么流水线？

官方 runtime.md 给了清晰的五层职责表：**Source 描述作品，Run 选择目标，Runtime 执行，Build 是一次执行尝试，Result 保存产出。**



核心是 Core 调度器：只管推进依赖，“并不知道自己在做视频”——新组件/新 Provider 用同一套包接口插进来

五层流水线（对应 docs/guide/runtime.md 的职责表）

仓库本体是约 **120 个包的 pnpm monorepo**：runtime-local（本地执行）、provider-hypihub（官方托管）、provider-whisperx-local（本地对齐）、render-hyperframes（无头 Chromium 并发渲染）等，全部围绕同一套接口。

Q6 生成模型是必须的吗？哪些东西不花一分钱也能渲染出来？

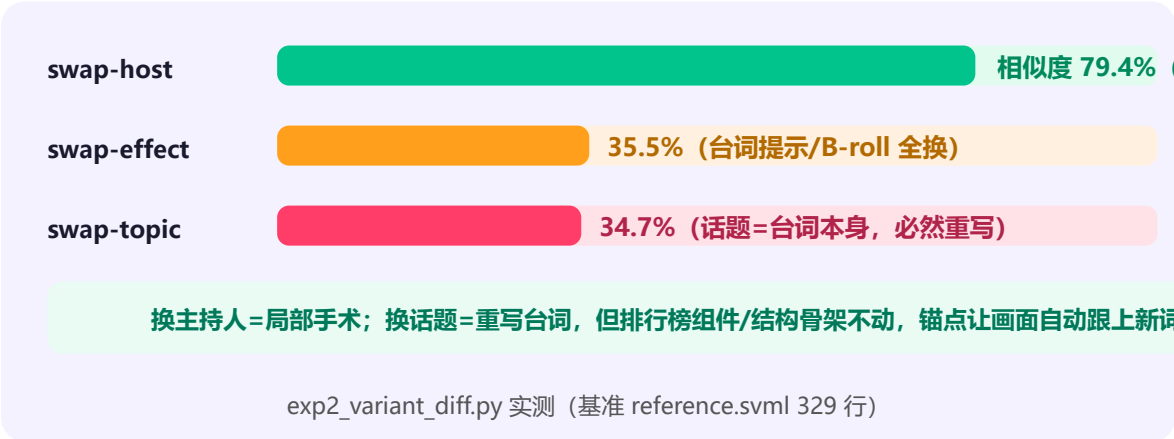
不是必须的——README 明确：**字幕、动效和代码渲染的画面，不调用生成模型也能编译成一条成片**。仓库把"要花钱的"和"本地免费的"拆成了不同的 Provider 包：

类别	包（源码实测）	花钱？
图像/视频生成	gpt-image、seedance、nano-banana、wan、pixverse、grok-image、seedream	按各家 API 计费
语音	elevenlabs-speech、fishaudio-speech、mimo-speech	按量计费
词级对齐	whisperx（可本地跑 provider-whisperx-local）	本地免费
组件画面	ranking、deck-track、caption、comment-sticker、interview-emoji-reveal...（代码渲染）	免费
渲染	render-hyperframes + 无头 Chromium（官方示例 64 并发进程）	免费（烧 CPU）
托管一条龙	provider-hypihub（HypiHub 一个账号全包）	托管计费

官方三个示例的真实成本：**GOAT DEBATE \$1.15 / DAILY CREATINE \$1.07 / NICE RIDE \$1.09**——一条 20 秒口播视频的生成成本约 1 美元，框架本身免费。

Q7 "换主持人只重新生成相关镜头"是真的吗？官方变体文件改动有多大？

官方在 examples 里放了三个"复刻变体研究"：swap-host（换主持人）、swap-topic（换话题）、swap-effect（换特效风格）。实验2对它们做了行级对账：



换话题的那次"大改"恰恰验证了架构价值：swap-topic 换成了马斯克话题（13 个新锚事件、全新提示词），但排行榜组件、字幕系统、外观包一行没动——新台词写完，画面元素自动跟着新词走，不需要重新掐时间。

Q8

hypit 命令行能干什么？check / measure / plan / build 分别是什么？

命令	干什么	要点
hypit version --check	查版本/更新	可执行体与 Skill 各有更新通道
hypit runtime init	生成 hypit.runtime.json 起始模板	只写配置，不装东西不登录
hypit check <svrun>	体检源文件/引用	构建前查错
hypit measure ... --segment ronaldo --pace fast	按语速策略估算台词时长	官方示例实测：两段各估 9s（快速 + 向上取整）
hypit plan	出执行计划+报价（Endpoint/费率）	付费前必经的一步：同意预算才开工
hypit build --follow	真正渲染成片	产出进 Build Results，过程可追问

注意 **measure** 和 **plan**：一个把"台词长度"变成"预估时长"（语义时间的外显），一个把"要花多少钱"变成"先报价后干活"——这两条都写进了官方 Skill 的生产纪律。

Q9

排行榜、聊天场景这些"组件"是怎么复用的？和 React 组件像吗？

非常像。每个组件是一个版本化包（如 @hypit/ranking@1），有自己的 props（传入画布、时间轴、字体、条目数据）和内部状态（TierBoard 会在论证进行时保持自己的状态——条目出现、移动、定格）。官方 examples 里有完整谱系：

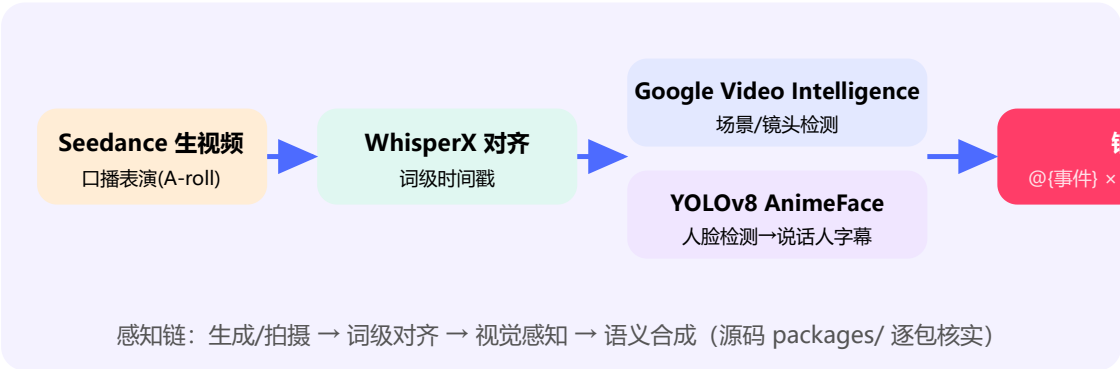
示例	组件亮点	语义时间用法
ranking-football	TierBoard 排行榜 + 双说话人 Take	条目在各自 Segment 的"不相交揭示窗口"里出现
semantic-composition	chat:Scene 聊天场景	4 条消息各自 at=0.5s/2s/3.5s/5.2s 挂在时间轴上
interview-emoji-reveal	人脸追踪 + 音效同步 emoji 揭示板	emoji 钉在关键词上
minimal-author-package	最小自研组件包骨架	学写自己的组件从这里开始

与 React 的类比：**props≈组件参数，时间轴≈render 循环，状态保持≈reconciliation**。区别是组件的"生命周期"由词级事件驱动而非浏览器事件驱动。作者还可以把组件打成 tarball 或发到私有 npm scope 分享——包管理那套直接搬过来了。

Q10

WhisperX、人脸检测在里面扮演什么角色？词级时间戳哪来的？

词锚定要落地，必须知道"每个词到底在哪一毫秒被念出来"。这条感知链路由几个本地/托管组件拼成：



这就是为什么复刻时参考视频要先"看懂"：WhisperX 给台词词时刻，视觉模型给镜头与脸的位置，Agent 再把"@{hair-gel}"这类锚点映射到新台词的新词上。**换配音、换语速、换语言后重跑一次对齐，全部画面事件自动重新踩点。**

Q11

许可证有什么坑？"开源"但不是纯 Apache-2.0？

LICENSE 实读： **修改版 Apache-2.0，附加三条条件**。

可以 	不行  (需商业授权)
自己组织内部商用、给客户做视频	拿源码运营 多租户 SaaS (两个以上外部用户各持独立 workspace 即算)
自己基础设施上单租户部署	商用转售 Hypit 或其衍生品 (单独或捆绑皆算)
fork+修改+开源发布 (同许可、不牟利)	去除/修改 CLI、运行报告、manifest 里的 LOGO 与版权信息

产出归你：许可证明确声明对你用 Hypit 制作的成片/音频/图片不主张任何权利，商用随意 (第三方模型服务另有各自主条款)。这个模式与 vault 已有概念[[内容型开源与双许可]]同族——"开放源码、约束商业模式"。

Q12

我想复刻一条爆款视频，最小上手路径是什么？要花多少钱？

第 1 步 装 Skill (一次性)： `npx skills add hypit-ai/hypit -g` ——装的是"导演知识+工具入口"，不需要克隆仓库。环境要求 Node 22.15+ / pnpm 10.33 / TS 5.9。

第 2 步 在 Agent 里带着参考视频开

□： `/hypit` 用这条视频做参考： `/path/to/video.mp4`，把产品换成我的，保留排行榜格式和开场能量

第 3 步 Agent 检查环境、要凭证：选 HypiHub 托管 (一个账号全包) 或自带 API Key / 本地模型。WhisperX 可本地。

第 4 步 看 plan 报价 → 同意预算 → `hypit build --follow` 渲染。参考价：20 秒口播约 **\$1 一条**。

第 5 步 拿到的不是一条视频，是 **一份可继续编辑的源工程**：换脸/换话术/换语言→批量出变体。也可以全程只用文字描述从零写 workflow，不用任何参考视频。

 **纯零成本路线 (本次实测采用)**：不装可执行体、不接生成服务，克隆仓库后用 examples/ 里的 SVML 源文件做解剖、对账、时间模拟三个实验 (见下)，把语言、架构、变体机制全部摸清——先理解再花钱。

这个项目给我什么 (能力清单)

能力	我能拿它干什么
读懂一套"视频源码"	以后看到 SVML/SVS/SVRUN 文件知道每一行在定义什么；别人的爆款工程可以直接拿来改
Agent 视频流水线	让 Claude Code/Codex 当导演：参考视频→ workflow→ 成片→ 批量变体，一条龙脚本化
词锚定设计思想	任何"多媒体跟着数据走"的场景都能借用：幻灯片跟演讲词、字幕跟播客、动效跟音乐节拍
组件化视频工程	排行榜/聊天/采访板子都是可复用包——自己写一个"股票行情板"组件挂进视频 workflow
成本可控的生成	免费层（代码渲染+本地对齐）先行验证创意，确认值得再做 \$1/条的生成；plan 报价前置，不会烧穿预算
合规边界清晰	自用商用随意、产出归自己；别拿去做 SaaS 卖就行（Q11 对照表）

动手实验（零依赖 · 全部真跑）

实1 SVML 解剖器：亲手拆一份"复刻爆款"的源文件 (exp1_svml_anatomy.py, 约 50 行纯标准库)

1. 怎么操作： `python exp1_svml_anatomy.py ../repo/examples/ranking-football/reference.svml` 目的：统计词锚事件/停顿/能力包/硬编码秒数 实现：用正则+标签计数把 SVML 拆成结构报告（注意：SVML 的 Script 文本允许裸 `<|>` 字符，标准 XML 解析器会报错——本身就是个方言知识点）

✅ 预期结果（真实输出）：锚点开启/关闭 8/8 配对 PASS、独立事件 8 个 (hair-gel、trophy、greek-god...)、停顿 22 个、读音可选 2 处、21 个 @hypit 官方包 + 1 本地包、**硬编码秒数 = 0**。对 swap-host/topic/effect 同样解剖：锚点数 8/13/7，随话题变化。

2. 改一个参数看变化：把脚本换成解剖 `examples/semantic-composition/chat.svml`——纯代码场景（无生成节点），看到的是另一种形态：4 条聊天消息各自挂在 `at="0.5s/2s/3.5s/5.2s"` 的动画时间轴上，说明 **Hypit 同时支持词锚定（口播）与秒锚定（纯动画），按需选择。**

实2 变体对账：换主持人/换话题到底改了多少行？ (exp2_variant_diff.py)

怎么操作： `python exp2_variant_diff.py ../repo/examples/ranking-football` 目的：用 difflib 对 reference 与三个官方变体做行级 diff 并按区块归类

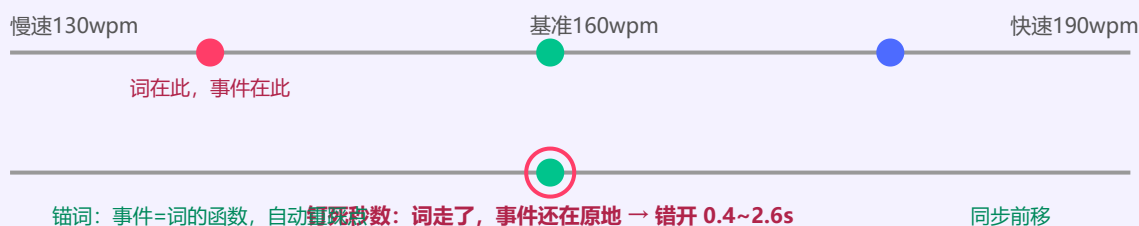
✅ **预期结果（真实输出）：** swap-host 相似度 **79.4%** (+49/-80 行，改动集中在 presenter 提示词与语音节点) = "换主持人只重新生成相关镜头"实锤； swap-topic **34.7%** (+223/-213, 台词/提示/B-roll 重写)； swap-effect 35.5%。三者的排行榜组件与结构骨架均保留。

中间可见的成果： 每个变体一张"改动区块条形图"——主持人相关 / B-roll / 排行榜组件 / 导入各占多少行，一眼看懂"哪里是手术、哪里是重写"。

实3 语义时间模拟器：语速一变，"锚词"和"锚秒"的命运分岔 (exp3_semantic_time.py)

怎么操作： `python exp3_semantic_time.py ../repo/examples/ranking-football/reference.svm1` 目的：把 ronaldo/messi 两段带锚台词按 130/160/190 wpm 三种语速展开成逐词时间轴，对比"事件锚词"与"事件钉死基准秒"的差异

✅ **预期结果（真实输出）：** ronaldo 段 37 实词 + 11 停顿 + 5 事件；语速 160→130wpm 时 @penaldo-social 从 14.40s 移到 17.00s、190wpm 时移到 12.62s。 **词锚定方案里事件永远踩在自己的词上；若钉死秒数，错开 0.4~2.6 秒——字幕弹在与词无关的时刻，一听就穿帮。**



实验3结论可视化：上=词锚定（事件跟词走），下=秒锚定（词走事件留）

诚实说明：本模拟器是简化模型（固定 WPM+固定停顿时长）；真实系统由 WhisperX 按实际音频逐词对齐，官方 hypit measure --pace fast 对同段台词估 9s，与简化模型的绝对值不同，但"事件随词移动"的定性结论一致。

一页带走

Hypit = 视频的"源语言 + 编译系统"：SVML 写作品（21 个能力包按需导入），台词里 @{锚点} 把画面钉在词上，Runtime Profile 决定花不花钱，Build 产出成片+可复用工程。复刻爆款的本质是 **Agent 读懂参考视频的"事件-词映射"，再用新词新脸重写同一套关系**——所以换脸只动脸、换话只动话，排行榜和字幕系统原封不动。三个实验（解剖 0 硬编码秒、变体对账 79% vs 35%、语速模拟 0.4~2.6s 错位）分别从源文件、工程、时间三个维度验证了"锚词不锚秒"不是宣传语而是架构事实。

三个真实的坑：① SVML 的 Script 文本含裸 <|>，标准 XML 解析器直接报错（方言自有词法）；② "已开源"是修改版 Apache-2.0——禁多租户 SaaS/商用转售/去 LOGO，别看见 Apache 就当无约束；③ 零成本路线只能验证"字幕/组件/代码画面"，带真人感的口播表演仍需生成模型（约 \$1/条）。